

**A New Finite-State System for
Modeling Sound Change, Applied
to Proto-Celtic**

2025

John Wang
Tübingen
Seminar für Sprachwissenschaft

Contents

Contents	2
List of figures	4
List of tables	5
Declaration of Academic Integrity	6
AI Usage	7
1 Introduction	8
2 Historical Linguistics	9
2.1 Neogrammarian Hypothesis	11
2.1.1 Limitations	12
2.2 Sound Laws	13
2.2.1 Definitions	13
2.2.2 Ordering	14
2.3 Computational Linguistics	16
2.4 Reframing the Neogrammarian hypothesis	17
2.5 Previous Study of Sound Changes	21
3 Finite State Transducers	22
3.1 Finite State Automata	22
3.1.1 Preliminary Definitions	22
3.1.2 Finite State Machine	24
3.2 Elementary Results in FSMs	26
3.2.1 Regular Expression	26
3.3 Finite State Transducers	28
3.4 The replace operator	30
3.5 Applications for FSTs	34
4 Infrastructure	36
4.1 CAPR	36
4.2 Project Description	40
5 Case Study Celtic	43
5.1 Background Information	43
5.1.1 Celtic	43

5.1.2 Proto-Indo-European	43
5.1.3 PIE Morphology	44
5.1.4 PIE Ablaut	44
5.1.5 Suprasegmentals	45
5.1.6 Phonology	45
5.2 The Celtic Dictionary	46
5.2.1 Sound Laws	46
5.2.2 Results	49
5.3 Discussion of Results	52
6 Discussion and Conclusion	53
Bibliography	55

Word count: 14443

List of figures

Fig. 2.1 A Basic Sound Law	13
Fig. 2.2 The Whine Wine merger	13
Fig. 2.3 Sound Law with context	14
Fig. 2.4 Grimm's law with features	14
Fig. 2.5 Great Vowel Shift for /a/	15
Fig. 2.6 American /æ/ tensing	15
Fig. 3.1 Example Automaton	25
Fig. 3.2 A Nondeterministic Machine	26
Fig. 3.3 The equivalent deterministic machine	26
Fig. 3.4 Obvious Approach	30
Fig. 3.5 An ambiguous law	31
Fig. 4.1 Capr Input View	37
Fig. 4.2 Capr Browse View	38
Fig. 4.3 Capr with error message	39
Fig. 4.4 Example Intermediate FST	42

List of tables

Table 2.1 Illogical English Spelling	10
Table 2.2 German Words	11
Table 2.3 Grimm's law	14
Table 3.1 Types of Replace operators	32
Table 5.1 PIE consonant table	45
Table 5.2 Dialectical Indo-European Changes	47
Table 5.3 Early Changes	47
Table 5.4 Late Changes	47
Table 5.5 Probable Changes	48
Table 5.6 Transduction table	49

Declaration of Academic Integrity

1. I hereby declare that I have written this work myself and have not used any sources or resources other than those stated. I have identified texts, ideas, concepts, graphics, etc. taken directly or indirectly from external sources as such and have provided complete references to the respective sources.
2. Use of generative AI:
 - a) **Option 1: Use of generative AI not permitted** I confirm that I have written this thesis completely autonomously, i.e. that I have not used any generative AI tools.
 - b) **Option 2: Labeling obligation if use of generative AI permitted** I am aware that the use of texts or content created by means of generative AI does not guarantee their quality and that I am responsible if the use of such tools results in incorrect content, violations of data protection law, copyright law, or academic misconduct (e.g. plagiarism). I also confirm that:
 - i. I have used generative AI tools as an aid only and that my creative influence predominates in the present work.
 - ii. I have indicated by exhaustive summary which generative AI tools I have used, for what purpose and to what extent.
 - iii. I have used only tools/resources in the manner explicitly permitted by the examiner(s) or the examinations office.
3. This work has not been submitted in the same or a similar form to any other examination authority in Germany or abroad.
4. I am aware that a violation of the above points may have consequences under exam law and, in particular, may result in the assessed work being graded as “fail” or the coursework being graded as “failed” and, in the event of multiple or serious attempts at cheating, may result in termination of my enrollment or the initiation of proceedings for the withdrawal of any academic title awarded.

AI Usage

I have used AI tools, specifically ChatGPT, to support various stages of this thesis with the aim of improving efficiency in routine or technical tasks.

- During the early stages of the project, I used ChatGPT to explore suitable libraries and architectural design ideas.
- While developing the implementation component of the project, I used ChatGPT for rapid prototyping of code snippets. All final code integrated into the project was rewritten manually. This was primarily applied to the web application component, which unfortunately played only a minor role in the final version of the project.
- During the writing phase of the thesis, I used ChatGPT to streamline the reformatting of tables and to assist with proofreading. The text was initially drafted by myself, and ChatGPT was subsequently used as a guide to enhance coherence, grammar, structure, word choice, and formality. All substantive content, arguments, and analyses are entirely my own.

Chapter 1

Introduction

This thesis presents rust-capr, a new finite-state historical linguistics tool for modeling sound change. While computational methods have been widely used in other aspects of computational linguistics, such as phylogenetics and cognate detection, much less attention has been paid to *sound laws*, the greatest resource in traditional historical linguistics. Previously approaches to modeling sound laws had been relatively isolated. Outside of academia, the constructed language community has experimented with developing systems to formally model sound change, but these are rather informal and relatively removed from linguistics research. CAPR [1] has recently demonstrated the promises of revisiting the traditional sound law approach. This thesis seeks to further develop this new area, by providing a new implementation for a system that allows linguistics to verify and refine their theories.

The broader context of this work is at the intersection of two subfields of linguistics: historical linguistics and computational linguistics. Historical linguistics has a long, traditional history that started over a hundred years ago. Computational linguistics, in contrast, only came into existence once computers were developed. There has been relatively little overlap in these two fields. This can be shown by the fact that even recent breakthroughs in historical linguistics have been facilitated not by tools crafted for the task, but by simple find-and-replace tools provided by a word processor. Computational linguistics on the other hand, has recently found success in areas like machine learning, and large language models. Despite showing impressive results, these areas are quite far removed from the more conservative historical linguistics. A larger problem is that these areas emphasize black box models, which prioritize results over understandable theories. This thesis seeks to explore an approach that unites these areas. The linguist still comes up with a theory, but the computer can speed up mundane tasks, and be entrusted to verify the linguist.

Chapter 2 gives background information on historical linguistics as a field. It describes the traditional approach to field, previous approaches in computational linguistics, along with goals that motivate this project.

Chapter 3 describes finite state transducers (FST), the fundamental structure used by rust-capr to model sound laws. Much background information about the theory of FSTs is given in order to explain the benefits and drawbacks of this FST approach.

Chapter 4 describes the tool itself. It describes CAPR, the tool which inspired rust-capr, and describes the motivation for this project. It then describes the tool, and what its original goals were, and what was accomplished by the tool.

Chapter 5 describes an application of the tool to reconstruct the sound laws from Proto-Indo-European to Proto-Celtic.

Chapter 6 discusses the results. It evaluates the success and limitations of rust-capr, and considers improvements for future work.

The code for this project can be accessed on Github [2], along with the web interface [3].

Chapter 2

Historical Linguistics

Language is not static: languages change. Not only do they change, but there are several aspects in which language can change. [4]

- Syntax: the order in which the words are placed. For example, English requires adjectives to come before nouns, but many romance languages require adjectives to come after nouns. Other languages, like Ancient Greek, allow for both orders. English is generally the verb come between the subject and object, but languages like Japanese require the verb to be at the end.
- Semantics: the meaning of words. Older generations may be aware of the change in meaning withing their own lifetimes. For example, *gay* previously just mean happy. Now *gay* is almost exclusively used to refer to homosexuality. The word *retarded* previously meant

slow, and was euphemistically used to describe an intellectual disability. It is now widely considered an offensive term.

- Morphology: words and their formations. For example, the King James Bible has *Thou shalt not kill* as one of the ten commandments. Yet, in modern English, *shall* does not change to *shalt*, but always stays *shall* no matter the circumstances. The word *whom* is the oblique form of *who*, but in everyday English, speakers almost always use *who* in those contexts.
- Phonology: the pronunciation of language. Most English speakers now merge the pronunciations of *which* and *witch*, but originally the *wh* had a different pronunciation from *w*. Some Americans retain the difference, such as Bob Ross.

Historical Linguistics is the study of these changes in languages. [4]

Historically, historical linguistics has mostly been focused on phonological change. For convenience, this thesis will also focus on this area of sound change, as it has the most developed theory which can be modeled by the methods which will later be discussed.

The evidence that language changes can be seen by simply examining the English language. The English language is notoriously known for its frequently arbitrary spelling rules, that is, its *orthography*. Table 2.1 shows a list of words that do not have the *k* sound, yet are spelled as if they began with a *k*.

Table 2.1: Illogical English Spelling

English Orthography	IPA transcription
knee	ni:d
knight	nart
knead	ni:d
knuckle	nʌkəl
know	noʊ

Learners of English orthography may have just accepted this as a fact of the English language. Some spellings just make no sense. However, it is possible to postulate reasons for such a spelling. The most compelling is due to language change. English spelling often reflects an earlier stage of the language, where words were pronounced differently. The spoken language changed, but the written language had failed to update to the newer pronunciation. The orthography can be seen as a fossil of the earlier stage.

This evidence is further supported when some words from German are examined, which happen to have similar meanings, shown in Table 2.2. In the German words, the <k> is still pronounced, unlike in English.

Table 2.2: German Words

English Word	German Word	Meaning
knee	Knie	knee
knight	Knecht	male servant
knead	kneten	to knead
knuckle	Knochen	bone

It can be theorized that German and English were once the same language. However, through the many years of being apart, various changes in pronunciation, grammar, and meaning have effected both languages, making them mutually unintelligible. One of these changes that happened relatively recently was the change shown in Table 2.1, of deleting <k> when before <n>, which happened in English, but did not happen in German.

Also, note that not only has the word initial <kn> cluster been simplified in the examples of Table 2.1, but it has disappeared throughout the entire language. In fact, native English speakers may even have trouble pronouncing the cluster in isolation. This is because the change has effected the entire lexicon, leaving not a single exception behind, meaning that the change was systematic, or *regular*.

This change was only one example of many that occurred in the history of the English language. English is not the only language that has changed, every living language has evolved from an earlier state.

2.1 Neogrammarian Hypothesis

As a child learning how to read English, seeing the spelling of the words listed in Table 2.1 might be generalized into a rule: <kn> gets pronounced as <n>. This generalization parallels the development of the development of English, so what initially seems to be a pronunciation rule may be more accurately described as a *sound change rule* or a *sound law*, a term introduced by the Neogrammarians, a group of German linguists from the 19th century. [4]

Additionally, this particular sound law is observable because of the conservative orthography of English. But English has only (relatively speaking) recently been written down. If, for instance, English and German had written down their language back when they were one language, would it be possible for a learner of the language to similarly form a large list of rules to derive the English pronunciation, and a different set of rules to derive the German pronunciation?

The Neogrammarians had a similar thought, and believed not only that English and German could be derived from a common ancestor like this, but that they could project even further back, relating most language of Europe along with many languages of India. These languages included English, German, French, Spanish, Greek, Russian, Hindi, and more. This is similar to Darwinian evolution. An ancient species, after millions of generations, can diversify into many different modern species. Similarly, many languages can be thought of as the descendants of an ancient language.

A scientist studying evolution is required to sequence DNA to understand the changes in a species. Just as DNA mutations drive the change in species, sound laws drive the change in languages. (This analogy is admittedly not perfect. DNA is a very complicated process that looks essentially random and without reason. Language in comparison, generally has rules that apply across the lexicon. Many languages also experience similar sound laws despite having no contact with each other, while species will rarely experience the exact same genetic mutation).

The oldest literary languages known to Neogrammarians: Latin, Greek, and Sanskrit, not only had correspondences in vocabulary, but also similarities in grammar. This led them to advancement in the reconstruction of the Proto-Indo-European (PIE) language, which is postulated to have existed in the past, but then diversified and split into various daughter languages. Similar to the example of English and German, often one language might have had a sound law apply, but another language might have preserved a more archaic form. [4]

The Neogrammarians created these sound laws under the Neogrammarian Hypothesis, as shown in Definition 2.1. [4]

Neogrammarian Hypothesis

Definition 2.1

Sound Laws are exceptionless.

In this, they tried to bring linguistics in line with the laws of the natural world. Just as the natural world is modeled by the language of mathematics, linguistics may be modeled by sound laws. Each sound law corresponds with a function from words to words. The full history of the phoneme structures of a language can be thought of a composition of these small functions.

2.1.1 Limitations

The Neogrammarian Hypothesis sought to make linguistics a science by emphasizing predictability and regularity. Like any science, however, the actual data will not be completely

deterministic from the model. They believed what exception that they encountered were merely sound laws that had been not sufficiently generalized. [4]

However this is probably not a very realistic model of language. Unlike the physical world, which has laws written in the language of mathematics that will predict exactly what will happen, languages are full of arbitrariness.

There are examples of changes that are *sporadic* and effect only one word. Other changes may display *nondeterministic behavior*, where a law has seemingly applied in a context for one word, but does not apply in another word which has the same context. This may even happen when these two words are the exact same, resulting in two descendants. For instance, the word *curse* has the descendant *cuss* which deleted the syllable final <r>. However, English has also preserved the unchanged form *curse*, which has essentially the same meaning. [5] Borrowings further complicate the issue, since they first follow the sound laws of the language it was borrowed from, and then follow the sound laws of the language it was borrowed to. [5], [6]

2.2 Sound Laws

2.2.1 Definitions

Linguists have a conventionalized representation for sound laws.

The most basic change is the arrow: $A > B$ where A is a phoneme that gets changed, and B is the phoneme that it gets changed into. Another notation that will be used in this thesis is shown in Fig. 2.1. As notational shorthand, it can be chained such as $A > B > C$, which can be viewed as two laws, $A > B$ and $B > C$.

$$A \rightarrow B$$

Fig. 2.1: A Basic Sound Law

Fig. 2.1 represents a sound law that happens regardless of the context it appears in. An example would be **whine-wine** merger, which can be more abstractly written as Fig. 2.2

$$\text{w} \rightarrow \text{w}$$

Fig. 2.2: The Whine Wine merger

However, it is more common that sounds change in only restricted contexts. This is exemplified in Fig. 2.3 [7]

$$A \rightarrow B / X \text{ ______ } Y$$

Fig. 2.3: Sound Law with context

where the variables A, B, X, Y can represent strings, or potential features. The dash can be viewed as the place in which the change happens. The variables X, Y represent the left hand side and right hand side contexts. These contexts frequently represent single phonemes, but could also represent classes of phonemes, or multiple phonemes concatenated together.

Phonetic features sometimes allow for a more succinct pattern in expressing a sound law, and may also reveal a more general pattern. For instance, Grimm's law is a law that happened in Germanic languages. It is why the stop series in Germanic frequently do not match with the languages in the PIE family. Written out completely, it is a sound change encompasses 12 smaller sound laws as shown in Table 2.3

Table 2.3: Grimm's law

$p \rightarrow f$	$b \rightarrow p$	$b^h \rightarrow b$
$t \rightarrow \theta$	$d \rightarrow t$	$d^h \rightarrow d$
$k \rightarrow x$	$g \rightarrow k$	$g^h \rightarrow g$
$k^w \rightarrow x^w$	$g^w \rightarrow k^w$	$g^{hw} \rightarrow g^w$

This can be rewritten in a more concisely using features as seen in Fig. 2.4

1. $\begin{bmatrix} + \text{ stop} \\ - \text{ voice} \end{bmatrix} \rightarrow [+ \text{ fric}]$
2. $\begin{bmatrix} + \text{ stop} \\ + \text{ voice} \end{bmatrix} \rightarrow [- \text{ voice}]$
3. $\begin{bmatrix} + \text{ stop} \\ + \text{ asp} \end{bmatrix} \rightarrow \begin{bmatrix} + \text{ voice} \\ - \text{ asp} \end{bmatrix}$

Fig. 2.4: Grimm's law with features

It is important to note that this is a rough convention, and lacks a full, formal definition, which all historical linguists would agree upon. The notation is inadequate to describe all types of sound law change. Authors will usually need to supplement this notation with English language explanations. However, for simple cases, it is possible to describe a sound change unambiguously.

2.2.2 Ordering

The laws mentioned in Section 2.2.1 are each relatively simple to describe. However, a sound law does not just occur by itself. In the history of a single language, there are often many

sound laws that have occurred. These sound laws can interact with each other, which forces an ordering onto the sound laws. This is called the *relative chronology*. [4]

There are two important terms that describe certain orders:

- *Feeding*: This is when the application of one sound law causes a second one to apply, that would otherwise *not* have applied.
- *Bleeding*: This is when application of a first law causes the second to *not* apply, that would have otherwise applied.

As an example, the Great Vowel Shift in English included the law shown in Fig. 2.5.

$$a \rightarrow \text{æ}$$

Fig. 2.5: Great Vowel Shift for /a/

Much later, the, /æ/ tensing occurred in (most dialects of American English) with the law shown in Fig. 2.6, further changed the /æ/ vowel before nasals. [8]

$$\text{æ} \rightarrow \text{ɛə} / \text{ ______ } [+nasal]$$

Fig. 2.6: American /æ/ tensing

The relation between these two laws is a feeding relation, causing the phoneme /a/ to change due to Fig. 2.5 and then again from Fig. 2.6. Thus, the word <sand> is pronounced [sɛənd]. If the ordering were reversed, the result would be different. Fig. 2.6 would not apply and only Fig. 2.5 would, making the outcome [sænd].

Feeding and bleeding make the historical linguist's job a bit more complicated. Not only must they provide proto forms and sound laws, but the order must be correct too. Also, when adding or hypothesizing a new sound law, the linguist must be careful that adding a new sound law does not incorrectly feed or bleed words that were previously correct.

Feeding and bleeding relations can be used to give a rough estimate of when a sound law took place. For instance, if it is known a word is attested at some time T that shows some sound change B and the modern language shows evidence a law A that fed B , then it could be inferred that A must have occurred also before T .

However, this is a very rough estimate. The set of sound laws for a certain language forms a partial order with the relation $\leq$ with $A \leq B$ if sound law A happens before B . This can be determined by looking at the feeding and bleeding relationships, but this partial order will usually not be total. Therefore, a reconstruction of the order of sound laws will often have multiple permissible orders.

2.3 Computational Linguistics

At first glance, the formal notation looks like a perfect fit for a computer program. Indeed, it is not hard to write a computer program that, given a set of words, and the sound laws that are known, to then write a computer program that transduces this for a small corpus of words. As early as in 1969, Raoul N. Smith created a program to test the validity of sound change laws that transform Proto-Indo-European into modern Russian. [9] In the same year, Stanton P. Durham and David Ellis Rogers created a program that generated Franco-Provençal dialect words from Latin. Others are discussed in Section 2.5. [9]

There are many benefits of this modeling the sound laws with a computer. First, it provides *verification* of the sound laws. Instead of relying solely on the linguist, who may make mistakes, the computer functions as a second eye to see whether the laws produce the expected outcomes. As the amount of sound changes is usually nontrivial, it is not hard to accidentally forget to apply a law, or apply it in the wrong order. A computational model will always compute the laws correctly and in the correct order, revealing inconsistencies between the laws the linguist proposes and the actual data.

In this light, the job of the linguist is like the job of a programmer. The sound laws are lines of code to be executed. The proto language is the input. The descendant language is the output. And the individual words are test cases.

Despite early explorations in using computers to model sound change, it has not been very popular, as linguists “lost interest in computerised forwards reconstruction just when the availability of ‘Find & Replace’ in word processing packages might have made it easy to accomplish without any programming expertise.” [9] Sims-Williams describes how he used this Find & Replace method to investigate the sound laws of Proto-Celtic. This is not as robust as a computer program. Microsoft Word did not have all IPA symbols available, so he had to use other ad-hoc symbols to replace them. Additionally, this approach would only reasonably work for a subset of the laws described in Section 2.2.1, as sound laws that used features would need to be transformed into characters. Despite these drawbacks, it was noted that “this low-tech workflow led him to discover previously overlooked cognates of well-known Indo-European roots and to identify errors in the traditionally assumed relative chronology” [1]. Even this simple verification using a word processor is often not used.

[the vast majority of historical linguists] continue to carry out the symbolic manipulations that constitute sound changes entirely in their heads.

— X. Gong, N. W. Hill, S. Knights, and J.-M. List [1]

More benefits of using a computer is discussed in the Section 4.2. In addition to the benefit of verification, a computer aids in reproducibility, and allows the user to quickly test orders that are constrained by feeding and bleeding relations discussed in Section 2.2.2.

Currently, more attention has been drawn to different aspects of computational linguistics, such as *phylogenetics*: quantifying how related languages or language families are. Much methodology in this area was inspired by algorithms in evolutionary biology. Another are is *cognate detection*: given a corpus of languages, can it be used to find potential words that are related? [10] Other approaches focussing on sound laws are discussed in Section 2.5, but they are not as popular today. This thesis seeks to follow the example of CAPR [1] that returns back to the Neogrammarian hypothesis.

2.4 Reframing the Neogrammarian hypothesis

The Neogrammarian hypothesis: *all sound laws are regular*, is a strong statement. Taken literally, the claim is unfortunately hard to uphold, once a little time is spent working with sound laws. Language is full of *irregularity*. There are very many ways in which sound laws will give the incorrect result when applying the sound laws without some leeway for correction.

The Neogrammarians were had already known that *analogy* was a common source of irregularities. An example of *analogy* is found in *paradigm leveling*. English has leveled its verb conjugation for most verbs, only adding the *s* to the third person singular:

1. *I eat*
2. *You eat*
3. *We eat*
4. *They eat*
5. *He eats*

But the verb *to be* has resisted this change just for the first person singular: *They are*, *We are*, but ***I am*** instead of the incorrect *I are*.

In Old English, there would have been a different verb form for all verbs in the first person singular. However, through paradigm leveling, many of the verb forms have now been “simplified” are now just the stem. If the sound laws were applied to these word form directly, without taking analogy into account, it would likely create many different outcomes instead of the ones that actually occur in modern English.

Borrowing also creates some potential problems with the Neogrammarian hypothesis. When a word is borrowed, it lacks the sound laws that would otherwise apply to the rest of the lexicon. For instance the German word *Computer* is borrowed from the English word *computer* which looks like it has had no sound laws applied to it, bypassing the other usual correspondences for native vocabulary.

Loan words can be especially troublesome with respect to the presence of *doublets*, which are words that have the same origin that coexist. In Romance languages, the most of the core vocabulary comes from Latin. In the natural process of time, it has undergone various sound changes. However, due to Latin being used as a prestige language, the original Latin word has been reborrowed. An example is the English words *hotel* and *hospital*, which were borrowed from Old French. They both originally stem from the same original Latin word *hospitalis*. [11] Any sound change system that tries to model this would require two different sound law paths. One which would require the inherited sound laws, another which would require less sound laws, that is more direct from the Latin.

Japanese provides another instructive case. Similar to Latin in the west, Chinese was a literary language used by the elites in Japan. As a result, Chinese words were continuously borrowed throughout the entire history. A Chinese character is not as transparent about its pronunciation as a Latin word, and the pronunciation was borrowed from various dialects and at different periods. As a result, the same Kanji would have multiple readings depending on the compound that it was in and how it was pronounced in the dialect it was borrowed from.

Even accounting for analogy and loans, sometimes sound changes do not have a predictable outcome. For instance, Mandarin Chinese is well known for not having stop codas, which used to constitute a tonal group in Middle Chinese. Due to Mandarin's phonology, where every syllable has to have a tone, the stop codas were distributed to other tones. However, the mapping from coda to tone is not completely predictable. This is exemplified in the words 不 (Mandarin pinyin *bù*) , and 一 (Mandarin pinyin *yī*) , which had previously both ended in /t/. (Compare the more conservative Cantonese pronunciations *bat1* [12] and *jat1* [13] .)

Their default tones are fourth tone and first tone, but depending on context, may switch to second tone and fourth tone. Thus, creating an algorithm to predict the outcome tone would sometimes be nondeterministic.

All of these exceptions pose challenges to the Neogrammarian Hypothesis. They imply that even if a program were designed so that all sound laws were applied faithfully, it would still fail in certain cases due to analogy, borrowing and sporadic changes. Previous effort in modeling both the sound changes and their exceptions proved to be too difficult. This explains why many explorations into computational methods for historical linguistics have focused

more tasks which require less supervision. They commonly use a general machine learning style workflow: data aggregations, train a model, then evaluate on human annotated work. This way, a machine learning model can be adapted from another field without a human having to learn the nuances of each particular language.

In contrast, building an algorithm by hand requires programming ability and linguistic expertise. Each case needs to be looked over, and the user must be able to debug when things do not go as expected.

Despite these problems, there is an advantage to using the Neogrammarian hypothesis. It is easy to mistake the strictness of the Neogrammarian hypothesis as a weakness, but it is instead be viewed as a strength. When a sound law fails to account for a form, there is a reason for it. Sometimes it is due to other phenomena such as analogy, borrowing or sporadic change. But sometimes it is because a theory needs to be refined. In this, it is parallel to other sciences. Theories can be wrong and fail, but they will have a reasoning as to why it failed. This is in contrast to machine learning models, which are generally black box models.

A common example is Grimm's law. The Latin word for "have" is *habeo*. On the surface, these two words have similar semantic meaning and phonetic structure. It might be logical to conclude that these two words are cognate. However, this fails to obey Grimm's law, which says that a germanic /h/ should be correspond with a Latin /k/. Indeed, the two are not cognates, but are just similar by coincidence. The true cognate of English "have" is Latin *capio*, which has a meaning of "take". Grimm's law rules out the false cognate, and provides a path to a correct one.

Without such laws, it is too easy to assign these two as cognates. And this also explains the hesitancy of traditional historical linguistics in adopting computer based approaches. A system that uses a machine learning like approach will require a simpler model of language, meaning a full sound law system is too complicated to adopt requiring simplifications such as context free changes. [14]

Machine learning models have been popular in computational linguistics partially because of similar work being done in computational biology. Language change can also be thought of as *mutations* similar to how DNA changes. These changes are essentially a model where all changes are sporadic. While this model may have been a valid assumption for DNA, this explicitly goes against the Neogrammarian hypothesis on regularity. Furthermore, purely statistical models often produce reconstructions that are opaque to linguists. Even if it correctly predicts the output, the reasoning may not be clear. Biologists work at the macro level, where statistical methods are required to create generalizations over many, many small mutations. It

is not feasible to examine each small mutation. Meanwhile, a linguist works at more of the micro level. Each sound change can be individually examined.

The strength of the Neogrammarian hypothesis is its explicitness. Rules can be checked and revised, and exceptions can be categorized and explained. Linguists need to build upon previous theories. An automatic system that has a theory without explanation requires the linguist to give that explanation. In this sense, the linguist must then reverse engineer the system's work, duplicating effort.

It could be better summarized that the language itself is in general not completely regular, but that a model with higher regularity is easier for linguists to work with. A linguist will find it easier to model rules that are regular, because this can be viewed as evidence. Under this assumption, we can also view how regular a theory is. For linguist, seeing where the sound law succeeds is just as important as seeing where it fails, and why it fails.

The original neogrammarian hypothesis can be revised to Definition 2.2.

Revised Neogrammarian Hypothesis

Definition 2.2

A sound law's usefulness is proportional to its regularity.

By relaxing the condition, some of the exceptional cases can be accounted for. Instead of viewing them as exceptions that break the model, they are just viewed as less useful sound laws, since they represent laws that *can* apply, but potentially do not. To account for this, any algorithm that models sound change will have to incorporate *nondeterminism*. Instead of saying that language change is a function that maps a word to a new word, it can be thought of as a function that maps a word to multiple words.

This new adaptation allows sound laws to be modeled computationally in a transparent way to traditional linguistics. By putting the focus back on sound laws, the focus of computational linguistics becomes realigned with traditional historical linguistics.

The biological analogy of evolution only partially corresponds to languages. In biology, DNA has to be passed from a parent to a child. This ignores the geographical nature of linguistics, in which languages share features due to them simply being in the same area as other languages. As supposed to the biological model, known as the *tree model*, the *wave model* suggests sound changes affect a geographical area, as opposed to assigning a parent and descendent. Previous computational historical linguistic work frequently assumes the tree model. The beauty of sound laws is that they do not depend on a tree or wave model: both can be modeled as just composition of sound laws.

2.5 Previous Study of Sound Changes

Despite their traditional role in historical linguistics, sound laws have received surprisingly little systematic study in computerized format. Hand coded sound changes are relatively rare.

One example of using computerized sound laws was explored by John Hewson [15] in the 1970s. After finding the corresponding sound laws in the Algonquin languages, he encoded these laws into a program. He then used the program to reverse the laws, taking as input words from the descendant languages and then outputting a proto Algonquin word. This program was then used to publish a dictionary of proto Algonquin. [10], [15]

In contrast, the constructed language (conlang) community seems to have taken a greater interest in the study of sound changes in comparison to historical linguistics. For instance, the Index Diachronica is a compilation of sound laws attested in natural languages around the world. [16] Its purpose is to provide conlangers with a reference for realistic sound changes to adapt for their artificial languages.

However, the Index Diachronica is not a very formal piece of writing, in the sense that a computer program can be automatically made from it. It is essentially a list, that likely requires a human to make inferences.

Conlangers have further developed a tool for forward applications, for example with the Sound Change applier by Zompist. [17] These allow users to input a proto-form and observe the output under a set of sound laws, verifying that their sound laws produce consistent results. This provides forward applications, but it does not provide backward application. This is suitable when the language is artificial like with the conlang community, but the backwards application is of more practical use for linguists. This is because they have different goals. The Conlanger seeks to create a history that parallels real world changes. The creator already has perfect knowledge of the proto-language, but needs a system to verify that the descendants have the correct outcome. The linguist knows the descendants, but needs a system to verify that their proto-forms are correct.

A formalism that offers convenient forwards and backwards constructions is FSTs, or finite state transducers. The formalism is discussed in detail in Section 3.3. The advantage of FSTs is that they are bidirectional: encoding a sound law from one proto language to a descendent gives the reverse direction for free.

A recent FST approach is CAPR [1], which will be discussed more detail in Section 4.1. CAPR is a program that uses FSTs to encode the sound laws that happened in a language.

The linguist uses a web interface to encode the sound laws via the XFST tool syntax, which then gets passed to HFST, a transducer library.

Chapter 3

Finite State Transducers

In this chapter, Finite State Transducers (FSTs) are introduced. However, in order to define FSTs and motivate their usefulness, some background information is required. FSTs can be viewed as a generalization of Finite State Automata (FSAs), and have important properties about their usage. Only the most important results will be listed here, and will be presented without proof.

Much of this chapter is adapted from R. M. Kaplan and M. Kay [18] .

3.1 Finite State Automata

Finite state automata (or finite state machine) are a mathematical model that can be used to model a variety of phenomena. The most relevant usage is in string recognition.

3.1.1 Preliminary Definitions

To talk about strings, a formal definition is required.

String

Definition 3.3

Given a set Σ a *string* can be defined recursively as being either

- ε , the empty string
- $c \cdot S$ where $c \in \Sigma$ and S is a string

The set Σ is usually called the *alphabet* and the members *characters*. The set of strings created from this Σ can be denoted as Σ^* . The $\cdot$ operator can be extended to a concatenation operator of two strings where

- $\varepsilon \cdot S = S$
- $(c \cdot S_1) \cdot S_2 = c \cdot (S_1 \cdot S_2)$

A string is a common example of a *monoid* because the $\cdot$ operator is associative, and ε is the identity. By convention, strings are usually put in double quotes and the $\cdot$ operator commonly omitted. For example, the strings “foo”, and “bar” are strings under the Latin alphabet. The string “Ελληνικά” is a string under the Greek alphabet. And “129893” is a string under the Arabic numerals. A string such as “abc” is the shorthand for full $a \cdot (b \cdot (c \cdot \varepsilon))$.

Now that strings have been formally defined, Definition 3.4 is a formal definition of *languages*.

Language

Definition 3.4

A formal *language* is set of strings.

Note that Definition 3.4 does not specify that the set must be finite, and under this definition, there are languages that have an infinite amount of strings.

Definition 3.4 can be used to define not only natural languages such as English, Spanish, and German, but also to define more artificial “languages”, such as all prime numbers represented in decimal.

As an abuse of notation, the $\cdot$ operator can also be extended to languages as shown in Definition 3.5

Concatenation of Strings

Definition 3.5

Let L_1 and L_2 be languages. Then $L_1 \cdot L_2$ is defined as

$$\{x \cdot y \mid x \in L_1, y \in L_2\} \quad (3.1)$$

Finally, Definition 3.6 defines the Kleene Star, another important operator.

Let L be a language. Then the Kleene closure L^* can be defined as the smallest set such that the following all hold

- $\varepsilon \in L^*$
- $L \cdot L^* \subset L^*$

3.1.2 Finite State Machine

A deterministic finite state machine can be defined as $(Q, s, \Sigma, F, \delta)$, where

- Q is the set of states
- $s \in Q$ is the start state
- Σ is the *alphabet*, a set of characters
- $F \subset Q$ the set of final states
- $\delta : Q \times \Sigma \rightarrow Q$ the transition function that maps each state to its next state when given a character

A finite state machine M can then be used to define a language L by stating that a word is in L if and only if M accepts L .

Every δ can be extended to $\delta^* : Q \times \Sigma^* \rightarrow Q$ function by defining

- $\delta^*(q, \varepsilon) = q$
- $\delta^*(q, c \cdot S) = \delta^*(\delta(q, c), S)$ for $c \in \Sigma$

Finally, a machine *accepts* a string S if and only if $\delta^*(s, S) \in F$.

When the amount of states in a finite state machine is small, it can be convenient to represent the FST as a graph. The states will be represented by the vertices of the graph, and the edges will be represented by the δ function, with a label for each character. A common convention for representing the final states is by drawing those states in a double circle.

As example, Fig. 3.1 shows a machine under the alphabet $\Sigma = \{a, b\}$ using this graph representation. It accepts a string if and only if the amount of “a”s in the strings is even.

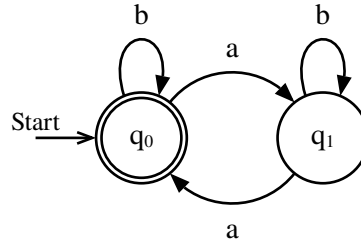


Fig. 3.1: Example Automaton

A related construct is the nondeterministic finite state machine or automata, also known as an NFA.

Nondeterministic finite state machine

Definition 3.8

A nondeterministic finite state machine can be defined as $(Q, s, \Sigma, F, \delta)$, where

- Q is the set of states
- $s \in Q$ is the start state
- Σ is the *alphabet*, a set of characters
- $F \subset Q$ the set of final states
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow P(Q)$, P being the powerset function. The nondeterministic transition function that maps each state and character to possible states.

Here, the transition function is different and can now map to multiple or zero states, in addition to just one state. Additionally, it can have ε transitions, with the intuition of not consuming parts of the string, while moving to a different state.

The nondeterministic finite state machine also has a similar accepts function.

Every δ can be extended to $\delta^* : Q \times \Sigma^* \rightarrow P(Q)$ function by requiring

- $\delta^*(q, \varepsilon) = \delta(q, \varepsilon)$
- $$\delta^*(q, c \cdot S) = \bigcup_{q' \in \delta(q, c)} \delta^*(q', S) \quad (3.2)$$

for each $c \in \Sigma \cup \{\varepsilon\}$, $S \in \Sigma^*$

Finally, an NFA *accepts* a string S if and only if $\delta^*(s, S) \cap F \neq \emptyset$.

A deterministic finite state machine is trivially a nondeterministic finite state machine. However, it may be surprising to know the converse is also true: any nondeterministic finite state machine can be turned into a deterministic finite state machine which accepts the same language, stated in Statement 3.9.

A language L can be recognized by a deterministic finite state machine F if and only if there exists a nondeterministic finite state machine F' that recognizes L .

Due to Statement 3.9, it usually does not matter whether it is called a deterministic finite state machine or a nondeterministic finite state machine, due to the set of languages they recognize being the same. In general, they can be referred as (FSM) finite state machines.

This can often simplify presentations in FSTs, due to nondeterministic machines often requiring less states. For instance, Fig. 3.2 is a nondeterministic machine that accepts strings that have a second to last character of “b”.

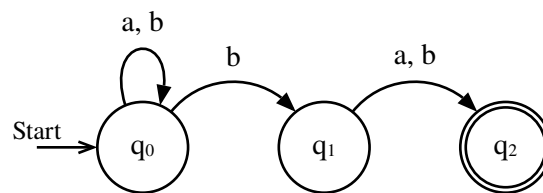


Fig. 3.2: A Nondeterministic Machine

Fig. 3.3 is an equivalent deterministic machine. It requires one more state and many more edges than Fig. 3.2.

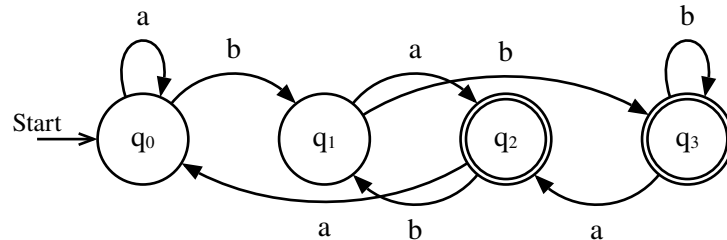


Fig. 3.3: The equivalent deterministic machine

When dealing with the theoretical properties of FSTs, whether the underlying machine is deterministic or nondeterministic can be relegated as an implementation detail. However, when dealing with real systems, the difference is more important, as determinizing a finite state machine can have a worst case performance of $O(2^n)$.

3.2 Elementary Results in FSMs

3.2.1 Regular Expression

Many programming languages have some sort of library for *regexes*, often used for string processing. The term *regex* is a shortening of the term *regular expression*, which Definition 3.10 defines.

Regular Expression

Definition 3.10

Given an alphabet Σ , a *regular expression* can be recursively defined as either,

- ε the empty regular expression.
- c where $c \in \Sigma$ the singleton character
- $R_1 \cdot R_2$ where R_1 and R_2 are also regular expressions, concatenation.
- $R_1 \mid R_2$ where R_1 and R_2 are also regular expressions, disjunction.
- R_1^* where R_1 is a regular expression, the Kleene Star.

A regular expression is not useful unless its semantics are also defined, as shown in Definition 3.11.

Regex Matching

Definition 3.11

Given a particular expression R , R *matches* a string S under the following conditions

- $R = \varepsilon$ and $S = \varepsilon$
- $R = c$, where $c \in \Sigma$ and $S = c$
- $R = R_1 \cdot R_2$ and there exists S_1 and S_2 such that $S = S_1 \cdot S_2$ and R_1 matches S_1 and R_2 matches S_2
- $R = R_1 \mid R_2$ and R_1 matches S or R_2 matches S
- $R = R_1^*$ and $S = \varepsilon$ or $S = S_1 \cdot S_2$ and R_1 matches S_1 and R_1^* matches S_2 .

Finally, the most important fact that relates regular expression with the FSMs in Section 3.1.2.

FSM and Regular Expression equivalence

Statement 3.12

Given a language L , there exists a regular expression R whose set of strings that match is L if and only if there exists an FST M such that the set of strings that M accepts is L .

In this case, L is called a *regular language*.

Some various properties can be proved for regular languages.

- Σ^* is regular
- If L is finite, then L is regular.

- If L is regular, then the reverse of L i.e. $\{\text{rev}(x) \mid x \in L\}$ where every string is reversed is also regular.

Special attention may be paid towards Statement 3.13, which will come up again in Section 3.3.

Recall the definition of $\cdot$ from Definition 3.5 and the Kleene star from Definition 3.6

Regular Closure properties

Statement 3.13

If L_1 and L_2 are regular, then $L_1 \cup L_2$, $L_1 \cap L_2$, $L_1 \setminus L_2$ and $L_1 \cdot L_2$ are all regular.

Also, if L is regular, then L^* is regular.

A natural question to ask is: are all languages regular? The answer is no. The pumping lemma describes the exact conditions in which a language is regular, but for brevity, it will not be described here. The most important example relevant to discussion is given in Statement 3.14.

Nonregular Example

Statement 3.14

Let the alphabet be the binary digits, i.e. $\Sigma = \{0, 1\}$, and let the language $L = \{0^x 1^x : x \in \mathbb{N}\}$. It can be shown that L is not regular.

This result can be generalized to alphabets beyond the binary digits. From this, it can be argued that many programming languages (which require parentheses to be matched like the example above) are *syntactically* not regular.

3.3 Finite State Transducers

The previously mentioned FSMs provide a convenient way of modeling phonology. Words can be defined as strings over some alphabet, such as IPA or simple ASCII characters. Under the assumption that natural language is finite, these languages would be finite.

However, *sound laws* describe not just how the languages are, but how they *change*. This motivates finite state transducers, which will be able to model the laws shown in Section 2.2.

A Finite State Transducer (FST or transducer for short) can be defined as $(Q, s, \Sigma, F, \delta)$, where

- Q is the set of states
- $s \in Q$ is the start state
- Σ is the *alphabet*, a set of characters
- $F \subset Q$ the set of final states
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times (\Sigma \cup \{\varepsilon\}) \rightarrow P(Q)$. The transition function

In Definition 3.15, the δ function now takes multiple characters from the alphabet. One can be viewed as the input, the other the output. Like an FSM, a convenient way of visualizing an FST is by viewing it as a graph. The vertices are Q and the edges are $(\Sigma \cup \{\varepsilon\}) \times (\Sigma \cup \{\varepsilon\})$. The edges are commonly notated using the notation $u : v$, where u is the input and v is the output.

A δ^* function can be defined as similarly, with the $\delta^* : Q \times \Sigma^* \times \Sigma^* \rightarrow P(Q)$ function by requiring

- $\delta^*(q, \varepsilon, \varepsilon) = \delta(q, \varepsilon, \varepsilon)$
- $$\delta^*(q, c_1 \cdot S_1, c_2 \cdot S_2) = \bigcup_{q' \in \delta(q, c)} \delta^*(q', S_1, S_2) \quad (3.3)$$

for each $c_1, c_2 \in \Sigma \cup \{\varepsilon\}$ and $S_1, S_2 \in \Sigma^*$

Each FST has two projections called the input and output projections by restricting the $\delta_{\text{input}}(q, s) = \bigcup_{s' \in \Sigma \cup \{\varepsilon\}} \delta(q, s, s')$ function and likewise for the output function, which can be used to create FSMs. Due to this, an FST can be thought of as a relation between two regular languages, i.e. a subset of $\Sigma^* \times \Sigma^*$. An input string is related to the output string if and only if the input FSM accepts the input and the output FSM accepts the output strings.

Such relation is called a *regular* relation.

Some properties of FSMs can be generalized to FSTs. Concatenation, union, and reversal have corresponding constructions in FSTs. Additionally, a very useful operator on two FSTs is the *composition* operator, shown in Definition 3.16.

Given regular relations R_1, R_2 , a pair (x, y) is in the composition $R_1 \circ R_2$ if and only if there is some z such that $(x, z) \in R_1$ and $(z, y) \in R_2$. In terms of sound laws, this corresponds with combining one sound law after another.

Note that the input and output constructions were chosen arbitrarily to be left and right. Indeed, it does not matter which is chosen to be which because given a regular relation R , there is the the inverse construction R^{-1} , which switches the input and the output, and is isomorphic to the original relation by $xR^{-1}y \Leftrightarrow yRx$. Note that this *inverse* construction, which can be constructed from an FST, should not be confused with the *reverse* construction, which refers to reversing a string. Although the names input and output are similar to the terminology in a function, in general, a regular relations does not need to be function, and can display nondeterministic behavior in the sense that one input could map to multiple outputs.

It was noted in Statement 3.13 that for regular languages, the compliment, union and intersection were all closed. There is a parallel statement given in Statement 3.17.

Closure Properties for Regular Relations

Statement 3.17

Given *regular relations* R_1 and R_2 , the relations $R_1 \circ R_2$, $R_1 \cup R_2$, and $R_1 \cdot R_2$ are all regular.

Additionally, for a regular relation R , then R^* is also a regular relation.

Notice that the *complement* construction and *intersection* are not mentioned in Statement 3.17. Indeed, they will not necessarily be regular. [18]

Consider the relation $R_1 = \{\langle a^n, b^n c^* \rangle \mid n \geq 0\}$ and $R_2 = \{\langle a^n, b^* c^n \rangle \mid n \geq 0\}$. The intersection of R_1 and R_2 is $\{\langle a^n, b^n c^n \rangle \mid n \geq 0\}$. Doing an output projection gives $\{b^n c^n \mid n \geq 0\}$, and as stated in Statement 3.14, this language is known to not be regular.

3.4 The replace operator

Using FSTs, it is relatively trivial to create a transducer that corresponds to an unconditional sound law. However, creating conditional sound laws is surprisingly nontrivial. A sound law describing $x \rightarrow y \mid a \text{ ______ } b$ is not as easy as it seems. The seemingly obvious approach shown in Fig. 3.4 does not work.

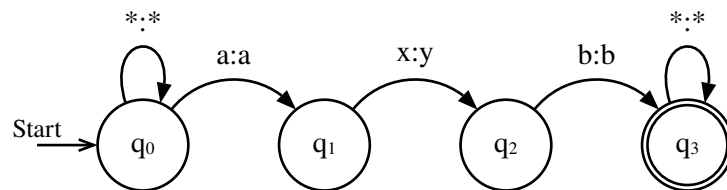


Fig. 3.4: Obvious Approach

Instead Fig. 3.4 only describes a rule will apply if there exists a context that can be replaced. For instance, a string such as “axb” would be correctly transduced to “ayb”. However, this is not the desired automaton. What is desired is an automaton that will replace all occurrences in all contexts. If there are no contexts, however, it should output the same string without changes. Fig. 3.4 would output no string if there were no appropriate contexts, such as a string like “ab”. Additionally, it fails to output the correct behavior when there are multiple contexts. A string like “axbaxb” would not output “aybayb”, but instead output “axbayb” and “aybaxb”, both of which are undesired.

Not only is such a simple case surprisingly complex, but there may be additional ambiguity when describing sound laws. For instance, consider the law in Fig. 3.5, what should be the output when transducing “xxxx”?

$$x \rightarrow y \ / \ x \ ______ \ x$$

Fig. 3.5: An ambiguous law

A reasonable answer is “xyyx”, but this poses a interesting situation. Due to each “x” being both a potential context and value to be transduced, certain heuristics may fail to hold. For instance, it may be appealing to have an approach that only transduces one character at a time, and then combine them together via the composition operator. This example shows the problem with that approach: a single transduction will destroy the context in which the second transduction should have applied. Therefore, in order to output “xyyx”, the transduction for both must happen at once.

Additionally, this breaks a symmetry that would otherwise appear. For instance, a context free law like $x \rightarrow y$, has a corresponding inverse rule $y \rightarrow x$ that will reverse the application. A rule such as $x \rightarrow y \ / \ x \ ______ \ x$ does not have a reverse of $y \rightarrow x \ / \ x \ ______ \ x$, since the string “xyyx” no longer has the same context in which the original rule applied.

This case seems to be artificial, but there are some linguistic phenomena that are similar. In general, this happens when the element that changes has itself as a context. One example is in Mandarin Chinese, where the third tone becomes the second tone when before another third tone.

R. M. Kaplan and M. Kay [18] discuss an approach to implement this behavior. One of the key insights with this approach is to first split it into smaller stages that are then composed together. One stage inserts *context markers* next to the left and right contexts (*a* and *b* in the example), which have a special symbol. A separate stage then transduces by doing the replace only between context markers. Finally, these markers are then removed in a separate stage.

L. Karttunen [19] uses a similar approach, and uses syntax that is also used in HFST [20] as shown in Table 3.1. This approach introduces four different ways of potentially disambiguating sound laws rewrites, since they can be ambiguous.

Table 3.1: Types of Replace operators

Upper Oriented:	UPPER -> LOWER LEFT _ RIGHT
Right Oriented:	UPPER -> LOWER // LEFT _ RIGHT
Left Oriented:	UPPER -> LOWER \\ LEFT _ RIGHT
Downward Oriented:	UPPER -> LOWER \ / LEFT _ RIGHT

To see the difference and where the ambiguity occurs, consider the example $a \ b \rightarrow x \ || \ a$
 $b _ a$, and applying it to the string *abababa*. This would yield *abxxa*, because the upper oriented requires that both left and right contexts match before it can transduce. However, the right oriented would yield *abxaba* and the left oriented would yield *ababxa*. The right oriented version requires the right context to match first before transducing, and the left context to match after transducing. Likewise, the left oriented version must have the left context match before transducing, and the right context must match after transducing. Finally, the downward oriented version is actually has two outputs: *abxaba* and *ababxa*, since they both satisfy the condition that the left and right contexts match after transducing.

The algorithm as presented in L. Karttunen [19] is a six part process consisting of

1. InsertBrackets
2. ConstrainBrackets
3. LeftContext
4. RightContext
5. Replace
6. RemoveBrackets

Applied to a general upper oriented replace pattern $UPPER \rightarrow LOWER \ || \ LEFT _ RIGHT$

Each step corresponds with a transducer. In the end, the steps are combined through composition to create the final transducer.

In the first part, *InsertBrackets*, takes as input any string, and as output, the same string, but with any number of bracket symbols (denoted *<*, *>*) interspersed at any arbitrary places in the output string.

The second part, *ConstrainBrackets*, is the identity transducer, except it disallows any occurrences of the substring “<>” to prevent a transduction of an empty string.

The third part, LeftContext, is a transducer that only allows as input strings which have each part that is part of a LEFT context marked with a the < context marker. In Karttunen’s notation, it is denoted as Listing 3.1.

```
~[~[...LEFT] [<...]] &
~[ [...LEFT] ~[<...]] ;
```

Listing 3.1: Karttunen’s Notation for LeftContext

Karttunen’s description in plain English is “The language in which any instance of < is immediately preceded by LEFT, and every LEFT is immediately followed by <, ignoring irrelevant brackets.”

This explanation due to a parallel of logical $P \rightarrow Q = \neg P \vee Q = \neg(P \wedge \neg Q)$, with the $\wedge$ operator here being a concatenation operator.

The fourth part is RightContext, is similar to Listing 3.1, but now for the Right instead of left.

The actual replacement occurs in the fifth step, which is shown in Listing 3.2.

```
%< UPPER/[%<|%>] %>
->
%< LOWER/[%<|%>] %> ;
```

Listing 3.2: Replacement in contexts that turns UPPER into LOWER

In Listing 3.2, it uses the -> operator, which is defined as as in Listing 3.3, given the expression UPPER -> LOWER.

```
[ NO_UPPER [UPPER .x. LOWER] ]* NO_UPPER ;
```

Listing 3.3: The -> operator

And in Listing 3.3, NO_UPPER denotes the complement of UPPER, but allowing for the empty string to also be in NO_UPPER. The .x. operator is the cross product of two regular languages, yielding a regular relation.

Consider, as an example, the law $x \rightarrow y / a \text{ ______ } b$.

In Listing 3.2, UPPER would be the regex that just accepts “x”, and LOWER would be the regex that only accepts “y”. In Listing 3.3, UPPER would be instantiated with a regex that accepts the string “<x>”, while LOWER would be the regex that accepts the string “<y>”. Taking the cross product of two (the .x. operator) creates an FST that only maps “<x>” to “<y>”.

Also, NO_UPPER would be the FST with an identity mapping that accepts any string that is not UPPER, i.e. anything that is not the string “<x>”. It is not hard to show that any string can be split into a concatenation of strings such that they each fulfil the following property: either it

is the string “<x>” exactly or it does not have “<x>” as a substring. These two cases correspond to [UPPER .x. LOWER] and NO_UPPER terms in Listing 3.3, concatenated by the Kleene star. Thus, this FST allows any string as its input.

Applying this to the string “axb” as input, the first few steps would cause the string to turn into “a<x>b”. The Kleene star in Listing 3.3 would apply only one time, yielding NO_UPPER [UPPER .x. LOWER] NO_UPPER. Finally, it would match the “a” into the first NO_UPPER, “<x>” into the [UPPER .x. LOWER] and “b” into the last NO_UPPER. Since the NO_UPPER all have identity outputs, and the [UPPER .x. LOWER] term has LOWER as its output, they would output “a”, “<y>”, “b”, combining into “a<y>b”.

Finally, the last operator is to remove the context markers that were previously introduced in the first steps. This is just the inverse of the first machine.

Karttunen also gives instructions in how to create each of the orientations mentioned in Table 3.1 by changing the order in which the composition is performed.

Note that this replace operator along with the four orientation is still not adequate to describe all changes that can be modeled with a program such as HFST. A crucial generalization is the parallel replace operator. The replace operator described here can be used to model the feeding and bleeding phenomena described in Section 2.2.2. However, it certain changes may be modeled as if they did not interact with each other. The current replace operator has to be composed in a serial composition, but for two laws to not interact with each other requires them to be composed in a parallel way. An example could be swapping two phonemes without introducing a temporary third symbol. Composing them serially will always cause one phoneme to be absorbed into the other one.

To account for parallel replacement, a generalized operator is required, which is described by A. Kempe and L. Karttunen [21]. Although the current implementation of rust-capr does not include this functionality, future work could expand the system to incorporate such an operator.

3.5 Applications for FSTs

The most influential finite state toolkit for linguistics was by the Xerox Finite State Tools (XFST). This tool demonstrated how finite state transducers could be applied to linguistic problems, especially in morphological analysis. [22]. This established not only practical usage of FSTs, but also the syntactic convention that later toolkits would adopt. However, a lim-

itation of XFST is that it is not open source. This has encouraged the development of modern alternatives that replicate and extend its functionality.

One of the most widely used open source FST libraries is OpenFST [23]. OpenFst is a relatively low-level library. Users can construct FSTs like graphs, specifying states and arcs explicitly. It expands the basic definition as defined in Definition 3.15 by supporting weighted transducers. This weight can take the form of any *semiring* (any set equipped with multiplication and addition).

OpenFst also provides the closure properties mentioned in Statement 3.13, allowing the user to use the closure properties in addition to constructing a graph based layout. This allows the user to specify the FST like a regular expression, as long as the FST is isomorphic to a finite state automaton. For those that are not, the complement and intersection operators are not available.

OpenFst is also not limited to morphological analysis. It has found usage in speech recognition [24] and other machine learning algorithms.

Rustfst [25] is a tool based off of OpenFst but written for the Rust programming language. It largely mirrors OpenFST's architecture, while also taking advantage of Rust's strong typing and memory guarantees, while generally maintaining the speed of OpenFST's C++ implementation. Rust also has decent support for WebAssembly through its LLVM backend, making it a convenient language to replace C++. Rust-capr is heavily based off of this library.

Foma [26] is a newer open source FST library that is written in C. Unlike OpenFST, it is a higher level library. It is made with a similar goal as XFST, which is morphological analysis. Its syntax is very similar to XFST and is mostly compatible with it.

One of the currently most used libraries for morphological analysis is HFST (Helsinki Finite State Technology) [20]. It is a toolkit written in C++ that integrates multiple backends. It uses OpenFst [23] as a potential backend and can also use FOMA as a backend. Its syntax is also compatible with XFST and FOMA, which allows for easy use of morphological analysis.

For rust-capr, these tools provided references for the design and implementation. HFST provided a reference for its algorithms, and rustfst was used as the implementation. As the HFST source code is available on GitHub [27], it was particularly valuable in the design of its algorithms, though rust-capr diverges by attempting to interface through the web interface as opposed to a command line tool.

This project was initially conceived as a highly experimental framework, prioritizing innovation as opposed to reliability. With this in mind, Rust as a language was chosen as the lan-

guage of implementation. Although C++ would have greatly simplified integration, simply by allowing direct usage of HFST, Rust offered greater potential by being a modern systems language. Its safety guarantees, strong typing, and modern tooling generally lead to a cleaner and more maintainable developer experience. The ease of compiling to WebAssembly also allow easier integrations with web applications.

Unrelated to rust-capr, there have also been FST approaches that have been developed independent of the XFST tradition, such as rsca [28], the reversible sound change applier, which was a C++ program that used FSTs to apply sound laws. However, this program seems to be unmaintained, and has not been as popular as the XFST family.

Chapter 4

Infrastructure

4.1 CAPR

This project was initially inspired the CAPR [1] system. It provides an innovative approach to using FSTs to model historical linguistics.

Unlike some existing work, CAPR does not attempt to learn correspondences or proto-forms from a training set. Some previous approaches such as the Bouchard-Cote [29] attempted to use statistical methods automatically come up with the sound changes that occurred. Other approaches, such as Hewson’s work in Algonquin languages, [15], put the responsibility on the linguist to manage all the programming, but are not meant to be general tool, being idiosyncratic to a specific language family. Capr seeks to create a blend of these two. It is general purpose system that can be adapted to any language while also not burdening the user with too many programming details.

The goal is not an automation system, but a creation of a verification tool. The linguist begins with a data set, such as a cognate set, and formulates hypotheses about the relationships between the languages. They then propose proto-forms and define sound laws that will derive the attested forms from these proto-forms. The system evaluates these hypothesis, and determines to what extent the proposed sound laws can account for the observed data.

Unlike other computational approaches, which often are partially “black box” models, the CAPR system mirrors a system closer to the traditional pen and paper process of the comparative method. (In comparison to another common approach of reconstruction, internal reconstruction. However, given previous usages of FSTs in morphology [30], it might not be hard to develop a similar approach for internal reconstruction). A theory is developed incrementally: initial hypotheses may be incorrect, but will be gradually refined.

More concretely, Capr is implemented as a web application. The linguist begins by supplying a dataset of cognates and defining the sound laws using the XFST syntax. Each law must explicitly specify which descendant languages it applies to, since Capr requires at least two descendants to reconstruct proto-forms. These rules are transmitted to a backend server, where they are compiled into finite-state transducers using HFST. The system then conducts backward translations for each cognate set. As outlined in Section 3.3, this can produce multiple candidate proto-forms. When the same proto-form emerge independently across multiple reverse transductions, this convergence serves evidence that the sound laws are internally consistent, and the corresponding results are presented to the linguist.

The screenshot shows the Capr web interface. On the left, there is a text area for inputting HFST syntax. The syntax includes definitions for initial consonants and medial consonants, such as `define pbInitC [ b:{*b} | p:{*p} | {?p}:{*7p} | m:{*m} | {?m}:{*7m} |`. On the right, there is a table titled "Initials" showing the results of the reconstruction for the initial consonant `k`. The table has three columns: "Gloss", "Maru", and "Bola". The rows show the reconstructed forms for the initial `k` in the words "persimmon", "umbrella", "cover / hide from view", "dance", "good", and "man". The results are as follows:

Gloss	Maru	Bola
"persimmon"	<i>ka³¹</i>	<i>ka²⁵</i>
*ga?	✓ *ga	✓ *ga
*ga?	✓ *ga	✓ *ga
"umbrella"	<i>ka³¹</i>	<i>ka²⁵</i>
*ga	✓ *ga	✓ *ga
*ga	✓ *ga	✓ *ga
"cover / hide from view"	<i>ka³¹</i>	<i>ka²⁵</i>
*ga?	✓ *ga	✓ *ga
*ga?	✓ *ga	✓ *ga
"dance"	<i>ka³¹</i>	<i>ka²⁵</i>
*gaX?	✓ *gaX	X *ga
*gaX?	✓ *gaX	X *ga
"good"	<i>kai³¹</i>	<i>kai²⁵</i>
*gai?	✓ *gai	✓ *gai
*gai?	✓ *gai	✓ *gai
"man"	<i>kai³¹</i>	<i>kai²⁵</i>
*gai?	✓ *gai	✓ *gai
*gai?	✓ *gai	✓ *gai

Fig. 4.1: Capr Input View

In Fig. 4.1, a screenshot is given of the Capr web interface. The left side shows the input of the sound laws, given in HFST syntax. The right side displays the corpus of words. For each proto-form, it reports whether the transduced result matches the data in the corpus.

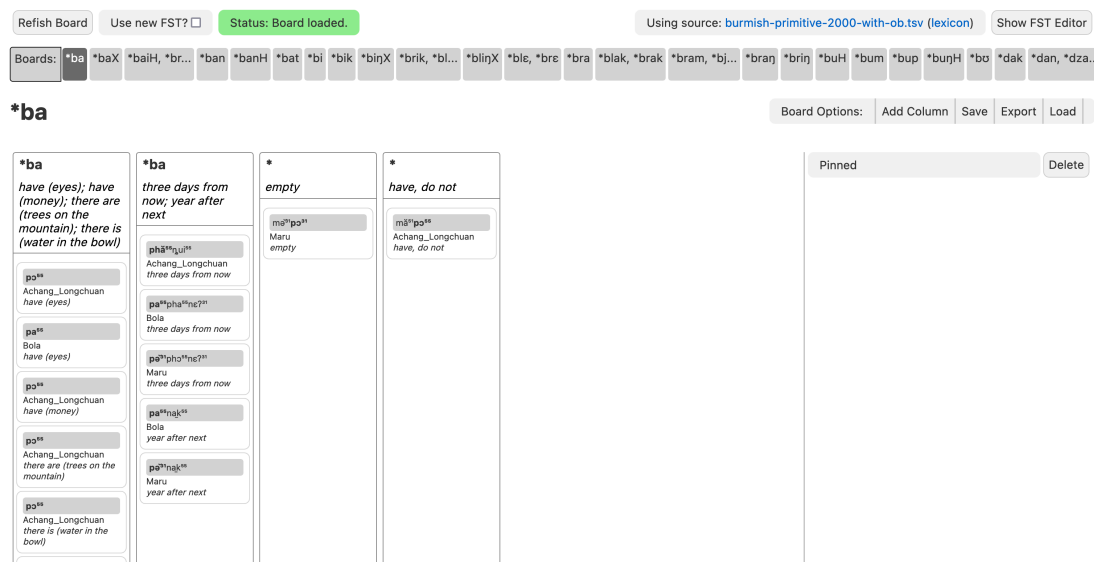


Fig. 4.2: Capr Browse View

In Fig. 4.2, it shows a browsing view for each proto-form that was generated. The linguist can also attach notes and organize the proto-forms.

The core idea of CAPR is very promising, with respect to it being an advancement in reproducibility and accessibility for those without a technical background, however there were some limitations that motivated this project.

First was general inconvenience. Installation required git, docker, and npm: tools that generally only those with a technical background would be familiar with. The setup instructions were also confusing, as the README.md had slightly different instructions compared to the SETUP.md file. The project also has a separate docker files for development and production. Confusingly, the production file requires the user to build the client themselves. Once docker finishes building the server and the user has build the client, they can navigate via the browser to the port mentioned in the terminal. Unfortunately, there appears to be a bug in the code, or it is not as robust, as instead of the nice view shown in Fig. 4.1, the view is shown in Fig. 4.3, which has a vague error message at the top.



Fig. 4.3: Capr with error message

Second, the dependence on HFST potentially introduces inconveniences. Since rule compilation is handled server-side, the linguist must either download the full project locally and building, or there has to be a remote server running to handle incoming requests. Both options come with additional infrastructure and maintenance costs. A simpler, self-contained web app that runs in the browser would improve accessibility for linguists and be more portable.

Third, the user interface requires the linguist to learn HFST. Although not particularly difficult, the syntax is different from the typical hand written form, creating an additional learning curve. Moreover, the linguist works mostly by typing in the edit box, which, although functional, is cumbersome. There is no syntax highlighting or syntax checking, making an already unfamiliar task more difficult. A more interactive interface could make the system easier to use and provide a smoother entry point for linguists.

Finally, Capr requires a strict adherence to the Neogrammarian Hypothesis. While this is consistent with classical historical linguistics, it limits the system's ability to model irregularities. A more flexible architecture would allow user to relax or enforce the Neogrammarian strictness as needed.

Despite these drawbacks, the CAPR project makes a lot of headway by opening a new direction in the field of computational historical linguistics that has been understudied. It improves upon earlier approaches by shifting the focus from opaque, data driven inference toward a more explicit modeling of sound laws as a verifiable object. This transparency also facilitates collaboration: linguists who are not specialists in a particular language family can independently inspect and verify the sound laws easily without needing to learn the language. In this way, Capr is unprecedented in this potential.

4.2 Project Description

The project is open source and hosted on GitHub [2]. In addition to the source code, a web-based interface is available through GitHub Pages [3].

This project serves as an exploration of lightweight transduction inspired by CAPR. Its primary objective is to investigate more user friendly approaches to sound-law transduction using finite-state transducers. The original vision was to create a system similar in spirit to CAPR but implemented in Rust, hence the name `rust-capr`. Although the final product remains a prototype in comparison to CAPR, it has yielded valuable insights and provides a foundation for future work.

Like its predecessor, `rust-capr` is not intended to automatically construct or predict word forms. Instead, it functions as a support tool for linguists, designed to automate the mechanical and repetitive aspects of transduction while leaving theoretical reasoning to the researcher. The linguist continues to formulate hypotheses about proto languages and their sound changes, while the tool assists in testing and verifying these ideas systematically.

The system fulfills two principal roles. First, it simplifies the process of formulating and applying sound laws, providing a framework for the verification of linguistic theories. In most dictionaries, etymological claims are presented without explicit validation. Even in dictionaries of proto languages, sound changes are not described rigorously. A mixture of formal notation and English are used, with the reader often needing to make some inference, rather than demonstrated systematically across all entries. `rust-capr` addresses this gap by explicitly modeling sound changes: each transduction can in theory be traced and inspected, rendering the derivation process transparent to both author and reader. In this respect, the tool also serves a proofreading function: the author proposes rules, and the system produces evidence for or against them.

The second role extends from verification to discoverability of proto-forms and sound laws. Once a theory is formalized, the system can automatically explore candidate proto forms that transduce correctly into attested descendant forms. This process leverages the bidirectional nature of finite state transducers. Even when inversion is not computationally feasible, heuristic search strategies could approximate candidate matches efficiently.

A sound law does not need to be perfectly accurate to be useful. Partially correct rules can still provide valuable insight, allowing linguists to identify the conditions under which a rule succeeds or fails and refine it accordingly.

Following these principles, the initial goal was to develop a minimal working prototype for historical linguistic reconstruction. The aim was to improve upon CAPR by reducing technical barriers for nonspecialists, simplifying the user interface, and adopting a more intuitive rule syntax than HFST. The envisioned design was a standalone, browser-based application requiring no installation. Although this goal was only partially achieved, the development process offered significant technical and conceptual insights.

The system’s core functionality is implemented in Rust, a modern programming language emphasizing high performance, memory safety, and modern tooling. Rust achieves C/C++ level speed without the overhead of legacy design constraints and uses compile time memory checks in place of a garbage collector, ensuring both reliability and efficiency.

rust-capr builds upon rustfst [25], a Rust port of the well-established OpenFst [23] library. While rustfst provides essential low-level FST operations, it lacks several higher-level features available in HFST. Furthermore, certain OpenFst operators remain unimplemented in rustfst, requiring custom reimplementations within this project.

A considerable portion of development was devoted to re-creating HFST functionality, most notably the replace operator discussed in Section 3.4, which is essential for context-sensitive rewrite rules. The implementation closely followed the HFST source code, which itself is based on Karttunen’s formalism. However, since the complement operator was absent in rustfst, it had to be implemented from scratch. This operator—frequently referenced in Section 3.4 as logical negation—is critical for the replace algorithm. Of the four orientations discussed in Table 3.1, rust-capr currently supports only the upper-oriented replace operator and does not implement parallel replacements, limiting it to one rule at a time.

Implementing the complement operator proved challenging. While the algorithm is conceptually straightforward, severe performance issues arose in practice. One key factor was rustfst’s use of weighted transducers. Although setting a state’s weight to zero should theoretically equate to deleting it, the library still treats it as active during computation. Because the determinization process can be exponential in complexity, even a single redundant state could trigger substantial slowdowns or apparent non-termination.

Developing the web interface also presented unexpected difficulties. Initial results were promising: simple rules executed quickly and interactively, but performance degraded with increasing rule complexity. While transduction itself remained fast (approximately $O(n)$ in relation to the number of states), FST construction became the main bottleneck. Build times scaled with the size of the symbol alphabet, which caused a large slowdown when working with the entire IPA. Although single character, non context based replacements compiled

rapidly, transductions that had either a character class in the left or right context increased build times dramatically.

Debugging added another layer of difficulty. Traditional debugging tools offer little help since an FST effectively represents a function rather than a sequence of instructions. The most informative visualization is a graph representation, but this becomes unwieldy once the number of states grows large. Figure Fig. 4.4 illustrates an intermediate FST generated during debugging. (The figure is a static screenshot, as embedding the full graph led to severe lag and occasional crashes.)



Fig. 4.4: Example Intermediate FST

Debugging is further complicated in that the problem was not necessarily problems with the logic, as it would frequently transduce properly, but problems with performance. Not only was it required to manually inspect many graphs like the one shown in Fig. 4.4, but time also had to be spent waiting for the program to create the graphs. This made the debug iteration process slow and frustrating.

Eventually efforts were mostly focused toward implementing a working back end transduction system.

The biggest issue was performance in general. Despite mostly following the HFST code as closely as possible, HFST significantly outperformed rust-capr's speed. Some bugs were caught that did improve rust-capr's performance, however, this still did not catch up to HFST's speed. Other factors may include the determinization algorithm being different in

rustfst's as opposed to the one used in HFST. In any case, after many debugging attempts, the root cause of the difference in performance was still unknown.

Chapter 5

Case Study Celtic

5.1 Background Information

The section provides some linguist background to understand the Celtic example. The objective here is to implement the sound laws that transformed Proto Indo European into Proto Celtic, and to evaluate how well these laws correctly reproduce the outcomes observed in the reconstructed Proto Celtic forms.

5.1.1 Celtic

The Celtic languages are a branch of the Indo-European language family. The living members of this family includes members such as Irish, Scottish, Welsh, and Gaulish. [31] The language which these language descended from is called Proto-Celtic. The term *Proto* is used because it has not been directly attested, and has been reconstructed from its descendants.

Celtic is in a unique position among the Indo-European languages in that it was previously widely spoken across continental Europe. However, the Celtic languages have died out in the mainland, and only the living descendants now are in the British Isles. What remains of the Celtic varieties are also becoming increasingly rare, due the dominance of English.

5.1.2 Proto-Indo-European

Proto-Indo-European (PIE) is the reconstructed ancestor of many languages of Europe. Its descendants include Germanic Languages (English, German, etc), Slavic Languages (Russian, Polish, Ukrainian), Romance Languages (Spanish, French, Italian, etc.), and many languages of India. Its study was the original study of the Neogrammarians. The grammar of PIE is reconstructed to be complex. The scope is quite large. It has a vast history and to go into full detail would take too long. Only the most relevant information will be covered here.

5.1.3 PIE Morphology

PIE was heavily inflected morphology. A root with a meaning could not be used by itself, but always had to have an ending. For nouns, they had to be declined according their case, number and gender. For verbs, it had to be conjugated according to person, number, tense, aspect, voice, and mood. In many daughter languages, this system was eventually simplified to various degrees.

Roughly speaking, nouns and verbs could be divided into two large inflection classes: thematic and athematic. The thematic declension has endings that begin with a vowel. The athematic declension generally had a stem that ended in a consonant and the ending was attached directly to the stem. The thematic declension is also somewhat more regular than the athematic class. There is also a tendency for the athematic class to simplify into the thematic class.

5.1.4 PIE Ablaut

PIE also exhibits forms of non-concatenative morphology. One phenomenon is the Indo-European ablaut. Roots all have the vowel *e*. Depending on the circumstances, such as if a root as certain suffix appended, or for certain conjugations and declension in the athematic class, this *e* can have different forms, termed grades. The *e* can turn into an *o*, called the *o-grade*. The *e* can sometime disappear, which is called the null grade, frequently denoted as $\emptyset$. Either the *o* or the *e* can also lengthen, giving the lengthened grade, denoted $\bar{e}$ and $\bar{o}$.

An example for ablaut can be found in the declension patern of the Ancient Greek word for “father”. The accusative case has the “πατέρα”, transliterated “patera”, showing the *e* grade, but the genitive case has “πατρός”, transliterated “patros”, showing the zero grade. [32]

Thus, certain words will descend from different grades. A word might have a certain grade in one language, but a different grade in a different language. A language might also simplify the ablaut, so it stays the same throughout the declension paradigm.

Most derivational morphology is done using suffixes, however, there is one example of an infix, which is the nasal infix, which gets inserted between the vowel and the syllable coda. English has one remnant of this pattern: “stand” has a nasal in its present tense, but its past tense is “stood” without an *n*. Another example is from Latin. The word for *leave*, which has as present “linquō”, but past tense “līquī” [33]. The present tense has its origin in PIE, “linékʷti”, which is built using the *n* infix from the root “leykʷ” [34]. Another common derivation strategy is reduplication, when the first consonant in the root is added to the front, with an intervening *i* or *e* in between.

5.1.5 Suprasegmentals

Proto-Indo-European exhibits suprasegmental features with its accent. The accent is generally unpredictable, and will change based the declension and declension patterns. For the thematic class, the accent generally stayed static throughout the paradigm. For the athematic class, the accent changed which syllable it landed on depending on the Certain families of Proto-Indo-European had changes based on the position of the accent. Proto-Germanic famously has Verner’s law, which explains apparent exceptions in Grimm’s law once accent is accounted for. Luckily, for Celtic, there is only one sound law that depends on the accent position, so for the most part the position of the accent was unneeded.

5.1.6 Phonology

The consonant phonology is given in Table 5.1. [35] The three consonants denoted h_1 , h_2 , h_3 are called the laryngeals. Almost all descendant languages lost these consonants, and there is much uncertainty as to how they were pronounced. However, they are reconstructable due to the effects they had when they were next to a vowel, changing the quality depending on the laryngeal. In descendant languages, they either disappeared or turned into vowels.

Table 5.1: PIE consonant table

Phoneme Class	Labial	Coronal	Palatal	Velar	Plain Velar	Labial Velar	Laryngeal
Nasals	m	n					
Stops (voiceless)	p	t		k	k	kʷ	
Stops (voiced)	b	d		ǵ	g	gʷ	
Stops (aspirated)	bʰ	dʰ		ǵʰ	gʰ	gʷʰ	
Fricatives		s					h₁, h₂, h₃
Liquids		r, l					
Semivowels				y		w	

Finally, PIE has interesting phonotactics. Phonologically, it only has only two true vowels, being the *e* and *o* which can ablaut. The others can be found as consonants that allomorphically function as vowels when there are not true vowels. The consonants *r*, *l*, *m*, *n* *y* and *w* therefore have their vocalic allomorphs *r*, *l̥*, *m̥*, *n̥*, *i*, and *u*. This complicates the reconstruction due to some descendants have different outcomes depending on if the surface form was a consonant or a vowel.

5.2 The Celtic Dictionary

The case study examines a Celtic dictionary by R. Matasović [31]. The dictionary begins with a short introduction, followed by a description of sound laws, and then finally the rest of the book consists of Proto-Celtic dictionary entries.

To test the validity of the Celtic dictionary, the sound laws have been computerized into the rust-capr library. For each sound law given in Section 5.2.1, Matasovic gives an example of a word that under goes that law. Each of these examples is added a list of Proto-Celtic words. The list of Proto-Celtic words were then expanded with arbitrarily chosen words from the dictionary. Each entry in the dictionary lists the original PIE form. It was tested if the original PIE form, after being applied to the sound laws mentioned, indeed match up with the expected Proto-Celtic form.

This tested not only the accuracy of the given sound laws, but if there were any consistent errors in the sound laws that could be further generalized into a better sound law.

5.2.1 Sound Laws

Given is a brief overview of the sound laws presented in the dictionary. They describe the changes from PIE to Proto-Celtic.

Table 5.5: Probable Changes

$f \rightarrow r / r \text{ ____}$
$f \rightarrow l / l \text{ ____}$
$r \rightarrow s / r \text{ ____}$
$s \rightarrow \emptyset / r \text{ ____ } t$
$m \rightarrow w / \text{ ____ } w$
$[-\text{long}] \rightarrow [+long] / \text{ ____ } x s [\text{liquid}]$
$ar \rightarrow ra / [+Labial] \text{ ____ } [+Dental] [+Dental]$
$al \rightarrow la / [+Labial] \text{ ____ } [+Dental] [+Dental]$
$[\text{laryngeal}] \rightarrow \emptyset / [\text{vowel}] y \text{ ____ } [\text{consonant}]$

Table 5.2 describe the earliest sound changes that had already started to appear in Proto-Indo-European. Table 5.3 describe laws that appear in early Proto-Celtic, and Table 5.4 describe changes that appear in later Proto-Celtic. Finally Table 5.5 describes changes that are probable, but the author is not as sure about how general they are.

These sound laws are generally in chronological order, however, there seems to be some inaccuracies that were found once these were tested.

In Table 5.2, it was noticed that there was conflict between the third law and the 6th law. If unmodified, the first law will bleed the sixth law, since resonants are also consonants and the laryngeal will no longer be there after the application of the sixth law. In order to correct for this, the implementation switched the ordering of these two laws so they would not conflict.

Also, in Table 5.2 there is one law that has stress, which introduced some problems in transduction. Seemingly obvious solutions, such as treating stressed vowels as different symbols as compared to their normal counterparts, and treating the stress marker as a symbol, each come with drawbacks. A system which treats the stressed vowels as different symbols would have to essentially make copies for every law that involves a vowel, one with the stress and one without the stress. A system that treats the stress symbol as a separate phoneme would be even more problematic. If a law changes the vowel has both a left and right context, adding the stress marker would cause it to no longer match the context. The system would have to ensure that the context would somehow be able to ignore the stress marker, or the system would have to duplicate the laws again to account for version with the stress versions included.

In Table 5.3, the seventh law presented difficulties. Although it is not too difficult to express the “...” as a Kleene star of consonants, compiling this sound law caused problems with respect to performance.

In Table 5.4, there is an immediate problem with the second law. As it is presented, it would be impossible to have the cluster “fl”, since “f” can only be created by the fourth rule by “p”, but a cluster of “pl” would have turned into a “bl” by the second law. However, there are many words in Proto-Celtic dictionary that begin with the “fl” cluster. The author likely meant for the second law to only apply word medially.

In Table 5.5, for the most part, the sound changes seem consistent. The only questionable part is the fact that the final law mentions laryngeals, but laryngeals should have disappeared in early Proto-Celtic. It is therefore unclear what the ordering should have been for when the last sound law occurred.

5.2.2 Results

Once these sound laws were formalized and turned into FSTs, a set of PIE etyma were then selected as test cases to see if the transduced result was the expected Proto-Celtic form. These words were more or less selected arbitrarily. Many words also required some slight modification, due to the Proto-Celtic entry containing a vowel as part of the declension, when the PIE root mentioned did not have it. Some words were also modified due to them coming from a different ablaut grade than the cited PIE root, or had a different suffix appended to it. Table 5.6 shows the results.

The code that was used to generate this table can be found in the repository under the tests folder. [36]

Table 5.6: Transduction table

Pie Stem	Transduced Result	Proto-Celtic Root	Correct	Notes
h ₃ eyno	oyno	oyno	✓	
b ^h ero	bero	bero	✓	
b ^h rso	barso	barso	✓	
deqno	da:no	da:no	✓	
dnt	dant	dant	✓	original danto
d ^h g ^h esi	xgesi	gdesi	✗	metathesis
d ^h g ^h o:m	xga:m	gdon	✗	metathesis
geh ₂ r	ga:r	ga:r	✓	original ga:ri
genh ₁ os	genos	genos	✓	
genu	genu	genu	✓	
grh ₁ no	gra:no	gra:no	✓	
g ^h yemo	gyemo	gyemo	✓	
h ₁ epirom	efirom	efirom	✓	
h ₁ lud ^h	lud	lud	✓	

Pie Stem	Transduced Result	Proto-Celtic Root	Correct	Notes
h ₁ re:g	ri:g	ri:g	✓	
h ₁ rewd ^h	rowd	rowd	✓	original rowdo
h ₁ su	su	su	✓	
h ₂ mlgto	amlixto	mlixto	✗	Incorrectly feeds <i>a</i> insertion
h ₂ ste:r	stir	stera:	✗	Incorrect vowel and extra <i>a</i>
h ₂ weh ₁ nto	winto	winto	✓	
h ₂ weh ₁ nto	winto	winto	✓	
h ₃ ektoh ₁	oxto	oxtu:	✗	Incorrect vowel
kapr	kabr	gabro	✗	initial consonant is irregular
kewh ₂ ro	kowaro	kawaro	✗	Incorrect vowel
kleh ₂ ro	kla:ro	kla:ro	✓	
klewos	klowos	kluwos	✗	Incorrect Vowel
klh ₁ eto	kleto	kaletto	✗	<i>a</i> inserted
kmti	kanti	kanti	✓	
krd	krid	krid	✓	
krdtu	krissu	krissu	✓	
krewh ₂	krow	kru:	✗	Incorrect vowel
kwo:n	kwa:n	k ^w on	✗	Incorrect vowel
k ^w end ^h	k ^w end	k ^w end	✓	so element at the end seems to be an addition kwendso
k ^w etwores	k ^w etwores	k ^w etwores	✓	
k ^w ey	k ^w e:	k ^w e:	✓	original kwe:s <i>s</i> includes the nominative ending
k ^w id	k ^w id	k ^w id	✓	
leg ^h	leg	leg	✓	original lego/ legyo
lewko	lowko	lowko	✓	
leyd	le:d	loydo	✗	incorrect vowel, may be from <i>o</i> grade
link ^w	link ^w	link ^w	✓	original link ^w o original leyk ^w , but <i>n</i> infix added
med ^h yo	medyo	medyo	✓	
mel	mel	mall	✗	Does not account for double consonant
meh ₂ k	ma:k	mak	✗	original mako, incorrect vowel length

Pie Stem	Transduced Result	Proto-Celtic Root	Correct	Notes
meh ₂ te:r	ma:ti:r	ma:ti:	✗	Somehow the <i>r</i> is dropped
mrg ^w to	mrixto	mrixto	✓	
neb ^h os	nebos	nemos	✗	Alternation between <i>m</i> and <i>b</i>
nem	nem	nem	✓	original nemo
newos	nowos	nowyo	✗	
newyo	nowyo	nowyo	✓	
neh ₂ u	na:u	na:wa:	✗	Due to switching from <i>u</i> to an <i>a</i> declension paradigm
nok ^w t	noxt	noxt	✓	
pelh ₁ u	felu	filu	✗	says it preserves <i>e</i> grade, but don't know where <i>i</i> comes from
ph ₁ te:r	fati:r	fati:r	✓	
piprqse	fibra:se	fibrase	✗	Vowel length
plew	flow	flow	✓	
plh ₁ no	fla:no	fla:no	✓	
potr	fotr	fatar	✗	cannot account for a vocalism
prptu	brixtu	frixtu	✗	Wrong initial
prptu	frixtu	frixtu	✓	
re:g	ri:g	ri:g	✓	
re:gnih ₂	ri:gni	ri:gani:	✗	Extra <i>a</i> epenthesis
reyd	re:d	re:d	✓	
reyd ^h	re:d	re:d	✓	original re:do
reyh ₁	re:	re:no	✗	<i>no</i> suffix
sed	sed	sed	✓	original sedo
seh ₂ l	sa:l	salano	✗	Incorrect vowel length along with <i>no</i> suffix
seh ₂ t	sa:t	sa:t	✓	original sa:ti
sek ^w	sek ^w	sk ^w etlo	✗	Some sort of metathesis with <i>lo</i> suffix
seno	seno	seno	✓	
sent	sent	sent	✓	original sentu
septm	sextm	sextam	✗	Fails to append add an <i>a</i>
seh ₂ g	sa:g	sagyo	✗	Wrong vowel length and <i>yo</i> suffix

Pie Stem	Transduced Result	Proto-Celtic Root	Correct	Notes
sih ₂ m _{do}	siando	sindo	✗	Law incorrectly feeds
sisk ^w o	sisk ^w o	sisku	✗	Failed to delabialize
skeh ₃ t	ska:t	ska:t	✓	original ska:to
skrib ^h	skrib	skri:bba:	✗	
slunk	slunk	slunko	✗	original slewk
smh ₂ el	smal	samali	✗	
sneh ₂	sna:	sna:	✓	
sney	sne:b	snig ^w	✗	Incorrect feeding rule
soru	soru	serwa:	✗	Incorrect vowel
spelh ₁ g ^h	sfelag	sfelga:	✗	Metathesis of vowel and consonant
sperh ₂ g	sferag	sfraxto	✗	Metathesis and <i>to</i> suffix
srewm	srowm	srowman	✗	n suffixed onto end
sterk	sterk	stronko	✗	<i>o</i> grade and metatesis
steh ₂	sta:	sista	✗	reduplicated
stomn	stomn	stamna:	✗	Wrong vowel with long a at end
supno	sowno	sowno	✓	
sweh ₂ du	swa:du	swa:du	✓	
sh ₂ l	sal	sal	✓	original saltro
tenh ₁	ten	torano	✗	A suffix of ro followed by a metathesis has been applied
terh ₁ tro	teratro	taratro	✗	Spurious <i>e</i> to <i>a</i>
treys	tre:es	tri:s	✗	Multiple vowels com- bining
tuh	tu	tu	✓	
uper	ufer	ufor	✗	Incorrect vowel, could be from <i>o</i> grade
wid ^h u	widu	widu	✓	
wirh ₁ o	wiro	wiro	✓	
wlh ₁ o	walo	walo	✓	
yemh ₁ o	yemo	yemono	✗	<i>no</i> suffix
g ^h eh ₂ ns	gans	gans	✓	original gansi
g ^h elh ₃	gel	gel	✓	original gelo
g ^w ow	bow	bow	✓	

5.3 Discussion of Results

For simple cases, such as when the word is the exact same in Proto-Celtic as in PIE, the system performs well, and usually transduces these without error. There are many cases also, where the tool almost gets it right, but the Proto-Celtic has an extended suffix on it. In these cases, they have been noted in the *notes* column if the a suffix has been appended.

One gap was identified: there was no sound law addressing so called “thorn” clusters in the PIE literature. These are the clusters “d^hg^h”, which are often prone to metathesis in daughter languages. Metathesis is when two phonemes switch positions. Compare the unmetathesized Tocharian B (tkam) vs the metathesized Ancient Greek (χθών *khthōn*) [37] .

Metathesis is another potential problem when dealing with sound laws, since often times they apply irregularly.

For the most part, there were not too many errors that were systematic. Many errors appeared to be unique to the word. Incorrect vowel outcomes appeared sporadically, while the consonants in contrast were somewhat more stable.

Chapter 6

Discussion and Conclusion

In Chapter 4, the project infrastructure and technical design were discussed. In Chapter 5, a proof of concept was presented that applied the developed tool to the transduction of Proto-Indo-European (PIE) forms into Proto-Celtic. The overall process of building the system was extensive and revealed many challenges, insights, and directions for improvement.

It may be noted that not all goals outlined in Chapter 2 were fully achieved. Working through the Celtic case study exposed several practical and theoretical drawbacks of relying on the FST formalism for this type of linguistic modeling.

One of the anticipated advantages of FSTs is their speed in performing transductions. Once a transducer has been compiled, it can operate extremely efficiently. However, this advantage

only holds after the FST has been built. The replace operator depends on the complement construction, which in turn requires determinization, an $O(2^n)$ operation. This leads to severe runtime slowdowns during compilation, especially for complex or large rule sets. To achieve acceptable performance, every FST must be minimized to prevent state explosion, but ensuring this consistently in practice is difficult, as minimization sometimes also is very slow.

Another major limitation encountered is that FSTs are hard to reason about and debug. The replace operator, as discussed in Section 3.4, relies on a somewhat unintuitive algorithm whose correctness is difficult to quickly verify. If a bug exists, it is often unclear where in the construction it arises. Unlike in typical programming languages, where one can step through execution using a debugger, an FST functions as a static graph. Although visualization tools can display the graph, they quickly become unmanageable once the transducer contains hundreds or thousands of states. Debugging therefore requires inspecting entire automata, which is time-consuming and cognitively demanding.

Runtime issues pose an additional challenge. Because determinization can grow exponentially, even a small inefficiency or suboptimal configuration can significantly degrade performance. Moreover, performance differences between HFST and rustfst suggest that implementation-specific factors such as differences in determinization algorithms may further contribute to inefficiency.

Consequently, the current version of the system still has drawbacks compared to HFST and the original Capr system. While the goal of a lightweight, installation free web application remains appealing, the performance limitations currently outweigh the convenience benefits. In this respect, an inconvenient system that works reliably is preferable to a convenient one that does not.

Beyond implementation challenges, there are also theoretical limitations to using FSTs for linguistic verification. One of the goals outlined in Chapter 2 was the verification of linguistic theories. The system can determine whether a proposed sound law fails, but not how or why it fails. For simple rules, viewing the resulting FST may suffice, but for complex systems, this becomes impractical. Ideally, a verification framework should not only output results but also “show its work” by tracing intermediate transformations in a transparent manner. Achieving this in an FST based system would require reconstructing intermediate states, which is computationally expensive and requires more infrastructure.

FSTs are also limited by their strong attachment to the string based representation of language. While strings are a convenient abstraction, many linguistic phenomena, particularly suprasegmental features like tone and stress, cannot easily be modeled in a purely segmental

framework. Even in the relatively simple Celtic example, modeling PIE accent placement posed difficulties, despite only one sound law depending on stress. Extending the system to multiple languages, as in the original CAPR project, would require a model capable of representing suprasegmental properties independently of phonemic strings. Naive strategies, such as treating accented letters as separate characters or encoding accent as a distinct symbol, introduce their own complications.

In hindsight, the choice of Proto-Indo-European as a test language also introduced additional challenges. PIE combines several phonological complexities, including movable accent, ablaut patterns, and the alternation between syllabic and nonsyllabic resonants. These make it particularly difficult to model computationally. The difficulty of FST based implementation appears roughly proportional to the complexity of a language’s phonology and phonotactics. The original CAPR paper used Burmish languages for its proof of concept, which have a simpler phonological system. Especially helpful is the fact that it lacks changes that potentially force resyllabification, making suprasegmentals easier to deal with. In comparison, PIE represents a nonideal and complicated scenario for testing.

In summary, the project has demonstrated both the potential and the limitations of using FSTs for computational historical linguistics. While in theory, the formalism provides rigor and interpretability, its technical constraints in performance and implementation suggest that future work should explore alternative approaches. The lessons learned from this prototype lay the groundwork for developing more efficient, interpretable, and linguistically expressive tools for modeling sound change and historical reconstruction.

Bibliography

- [1] X. Gong, N. W. Hill, S. Knights, and J.-M. List, “Computer-Assisted Approaches to Rule-Based Phonological Reconstruction,” 2023.

- [2] John Wang, *rust-capr*. [Online]. Available: <https://github.com/jzw2/rust-capr>
- [3] John Wang, “Rust Capr Web Interface.” [Online]. Available: <https://jzw2.github.io/rust-capr/>
- [4] H. Hock, *Principles of Historical Linguistics*. in Trends in Linguistics. Studies and Monographs [TiLSM]. De Gruyter, 2021. [Online]. Available: <https://books.google.de/books?id=uYdnEAAQBAJ>
- [5] “Wiktionary.” [Online]. Available: <https://en.wiktionary.org/wiki/cuss>
- [6] “Wiktionary.” [Online]. Available: <https://en.wiktionary.org/wiki/curse>
- [7] T. Crowley and C. Bower, *An Introduction to Historical Linguistics*. Oxford University Press, 2010. [Online]. Available: <https://books.google.de/books?id=RfpQEAAAQBAJ>
- [8] W. Labov, S. Ash, and C. Boberg, *The Atlas of North American English: Phonetics, Phonology and Sound Change a Multimedia Reference Tool*. Berlin: Mouton de Gruyter, 2006.
- [9] P. Sims-Williams, “Mechanising Historical Phonology,” *Transactions of the Philological Society*, vol. 116, no. 3, pp. 555–573, 2018, doi: 10.1111/1467-968X.12138.
- [10] P. Sims-Williams, “Mechanising historical phonology,” *Transactions of the Philological Society*, vol. 116, no. 3, pp. 555–573, 2018.
- [11] “Wiktionary.” [Online]. Available: <https://en.wiktionary.org/wiki/hotel#English>
- [12] “不.” Sept. 15, 2025. Accessed: Oct. 25, 2025. [Online]. Available: <https://en.wiktionary.org/w/index.php?title=%E4%B8%8D&oldid=86781275>
- [13] “—.” Oct. 25, 2025. Accessed: Oct. 25, 2025. [Online]. Available: <https://en.wiktionary.org/w/index.php?title=%E4%B8%80&oldid=87594393>
- [14] A. Bouchard-Côté, D. Hall, T. L. Griffiths, and D. Klein, “Automated reconstruction of ancient languages using probabilistic models of sound change,” *Proceedings of the National Academy of Sciences*, vol. 110, no. 11, pp. 4224–4229, 2013.
- [15] J. Hewson, *A computer-generated dictionary of Proto-Algonquian*. University of Ottawa Press, 1993.
- [16] “Searchable Index Diachronica.” [Online]. Available: <https://chridd.nfshost.com/diachronica/all>
- [17] “Sound Change Applier.” [Online]. Available: <https://zompist.com/sca2.html>

- [18] R. M. Kaplan and M. Kay, “Regular models of phonological rule systems,” *Computational linguistics*, vol. 20, no. 3, pp. 331–378, 1994.
- [19] L. Karttunen, “The replace operator,” *Finite-State Language Processing*, pp. 117–147, 1997.
- [20] K. Linden *et al.*, “HFST—a System for Creating NLP Tools,” in *Systems and Frameworks for Computational Morphology*, C. Mahlow and M. Piotrowski, Eds., in Communications in Computer and Information Science., Germany: Springer-Verlag, 2013, pp. 53–71. doi: 10.1007/978-3-642-40486-3_4.
- [21] A. Kempe and L. Karttunen, “The Parallel Replacement Operation in Finite State Calculus,” *Technical Report MLTT-021, Rank Xerox Research Center, Grenoble Laboratory*, 1995.
- [22] K. R. Beesley and L. Karttunen, “Finite-state morphology: Xerox tools and techniques,” *CSLI, Stanford*, pp. 359–375, 2003.
- [23] C. Allauzen, M. Riley, J. Schalkwyk, W. Skut, and M. Mohri, “OpenFst: A General and Efficient Weighted Finite-State Transducer Library,” in *Implementation and Application of Automata*, J. Holub and J. Žďárek, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 11–23.
- [24] D. Povey *et al.*, “The Kaldi Speech Recognition Toolkit.”
- [25] *rustfst*. [Online]. Available: <https://github.com/garvys-org/rustfst>
- [26] M. Hulden, “Foma: a finite-state compiler and library,” in *Proceedings of the Demonstrations Session at EACL 2009*, 2009, pp. 29–32.
- [27] *hfst*. [Online]. Available: <https://github.com/hfst/hfst>
- [28] “a reversible sound change applier.” [Online]. Available: <https://000024.org/rsca.html>
- [29] A. Bouchard-Côté, D. Hall, T. L. Griffiths, and D. Klein, “Automated Reconstruction of Ancient Languages Using Probabilistic Models of Sound Change | PNAS,” *Proceedings of the National Academy of Sciences of the United States of America*, vol. 110, no. 11, pp. 4224–4229, Mar. 2013, doi: 10.1073/pnas.1204678110.
- [30] K. R. Beesley and L. Karttunen, “Finite-State Morphology: Xerox Tools and Techniques,” *CSLI, Stanford*, pp. 359–375, 2003.
- [31] R. Matasović, *Etymological Dictionary of Proto-Celtic*. in Leiden Indo-European etymological dictionary series. Brill, 2009. [Online]. Available: https://books.google.de/books?id=YN_YPQAACAAJ

- [32] “Indo-European Ablaut.” Oct. 23, 2025. Accessed: Nov. 03, 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Indo-European_ablaut&oldid=1318410615
- [33] “Linquo.” Oct. 12, 2025. Accessed: Nov. 03, 2025. [Online]. Available: <https://en.wiktionary.org/w/index.php?title=linquo&oldid=87398603#Latin>
- [34] “Reconstruction:Proto-Indo-European/Linékw̥ti.” Sept. 24, 2025. Accessed: Nov. 03, 2025. [Online]. Available: <https://en.wiktionary.org/w/index.php?title=Reconstruction:Proto-Indo-European/lin%C3%A9kw̥ti&oldid=87170401>
- [35] “Proto-Indo-European Phonology.” Oct. 25, 2025. Accessed: Nov. 03, 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Proto-Indo-European_phonology&oldid=1318678529
- [36] John Wang, [Online]. Available: <https://github.com/jzw2/rust-capr/blob/master/tests/celtic.rs>
- [37] A. Kloekhorst, “Proto-Indo-European “Thorn”-Clusters,” *Historical Linguistics*, vol. 127, no. 1, pp. 43–67, Nov. 2014, doi: 10.13109/hisp.2014.127.1.43.